NOVA SCHOOL OF SCIENCE & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE

MEI - Master in Computer Science and Engineering

Internet Applications Design and Implementation

Week 4 - Persistence. Data Layer

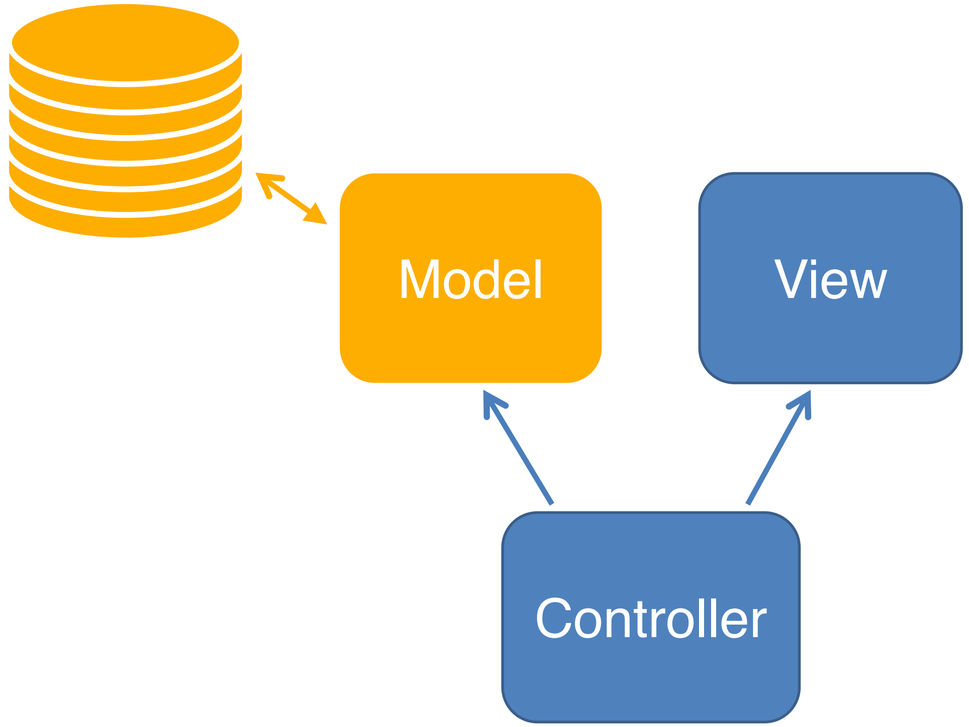
Information

Practical (labs) grades available in CLIP (Documents > Other)

This file will be updated regularly

© Pedro Alves 2026

Data Layer



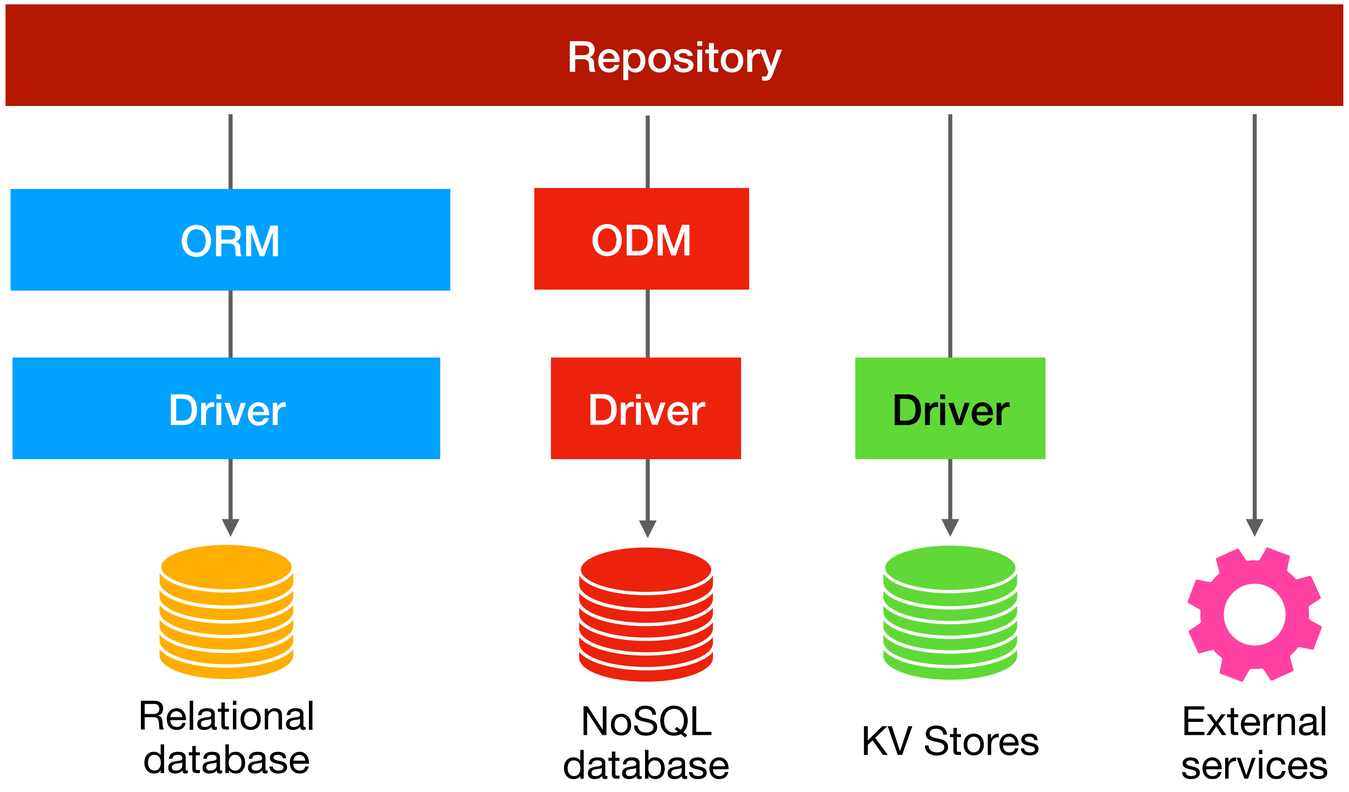
Data Layer

Abstracts how information is stored, related and protected

© Pedro Alves 2026

Data abstraction

Several levels of abstraction over the actual data model



© Pedro Alves 2026

Data storage model

Rank			DBMS	Database Model	Score		
Mar 2026	Feb 2026	Mar 2025			Mar 2026	Feb 2026	Mar 2025
1.	1.	1.	Oracle	Relational, Multi-model	1182.46	-21.05	-70.62
2.	2.	2.	MySQL	Relational, Multi-model	858.34	-9.88	-129.79
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	711.47	+3.33	-76.67
4.	4.	4.	PostgreSQL	Relational, Multi-model	680.08	+8.05	+16.66
5.	5.	5.	MongoDB	Document, Multi-model	383.58	+4.85	-12.85
6.	6.	6.	Snowflake	Relational	211.24	+3.10	+49.46
7.	↑ 8.	↑ 13.	Databricks	Multi-model	145.81	+1.29	+49.80
8.	↓ 7.	↓ 7.	Redis	Key-value, Multi-model	145.19	-1.85	-10.17
9.	9.	9.	IBM Db2	Relational, Multi-model	111.38	+0.16	-15.19
10.	10.	↓ 8.	Elasticsearch	Multi-model	103.58	-2.88	-27.80

https://db-engines.com/en/ranking

© Pedro Alves 2026

Data storage model

How is data stored

Relational databases

- Table-based model (rows and columns)
- Fixed schema defined before storing data
- Strong data consistency (ACID transactions)
- Relationships using foreign keys and joins
- Structured Query Language (SQL) for querying
- Vertical scaling traditionally (bigger machines)

Examples: PostgreSQL, MySQL, Oracle, SQL Server

NoSQL database

- Flexible data model (JSON, Key-Value, …)
- Schema-less or schema-flexible
- Often favor availability and scalability over strict consistency
- Typically support eventual consistency models
- Query mechanisms vary by database (JSON queries, map-reduce, custom API, …)
- Designed for horizontal scaling (sharding & replication)

Examples: MongoDB, Cassandra

© Pedro Alves 2026

NoSQL Querying

Example: IndexedDB

```
let transaction = db.transaction("myObjectStore", "readwrite");
let objectStore = transaction.objectStore("myObjectStore");

let data = { id: 1, name: "John Doe", age: 25 };
let addRequest = objectStore.add(data);

addRequest.onsuccess = function(event) {
  // Data added successfully
};

let getRequest = objectStore.get(1);

getRequest.onsuccess = function(event) {
  let result = event.target.result;
  // Access the retrieved data
};
```

© Pedro Alves 2026

NoSQL Querying

Example: MongoDB

```
db.orders.find({ status: "pending" })
```

```
{
   "_id": ObjectId("5f47a8ccef1f3b3dfb11b98c"),
   "clientId": 1,
   "orderDate": ISODate("2023-10-01T09:00:00Z"),
   "status": "pending",
   "items": [
      {"product": "Laptop", "quantity": 1},
      {"product": "Mouse", "quantity": 2}
   ]
},
{
   "_id": ObjectId("5f47a8cdef1f3b3dfb11b98d"),
   "clientId": 2,
   "orderDate": ISODate("2023-09-15T12:30:00Z"),
   "status": "shipped",
   "items": [
      {"product": "Monitor", "quantity": 1}
   ]
},
{
   "_id": ObjectId("5f47a8ceef1f3b3dfb11b98e"),
   "clientId": 1,
   "orderDate": ISODate("2023-10-05T10:15:00Z"),
   "status": "pending",
   "items": [
      {"product": "Keyboard", "quantity": 1}
   ]
}
```

© Pedro Alves 2026

Relational Querying in Java

JDBC - Java Database Connectivity

- Basic API for Java defining an access to a database
- DBMS agnostic - It works with any Relational Database
- Needs a JDBC driver for a specific DBMS (runtime dependence)
- Does not know the database schema
- Programmer needs to “manually” build SQL queries and handle database results

```java
try (Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery( "SELECT * FROM MyTable" )
) {
    while ( rs.next() ) {
        int numColumns = rs.getMetaData().getColumnCount();
        for ( int i = 1 ; i <= numColumns ; i++ ) {
           // Column numbers start at 1.
           // Also there are many methods on the result set to return
           //  the column as a particular type. Refer to the Sun documentation
           //  for the list of valid conversions.
           System.out.println( "COLUMN " + i + " = " + rs.getObject(i) );
        }
    }
}
```

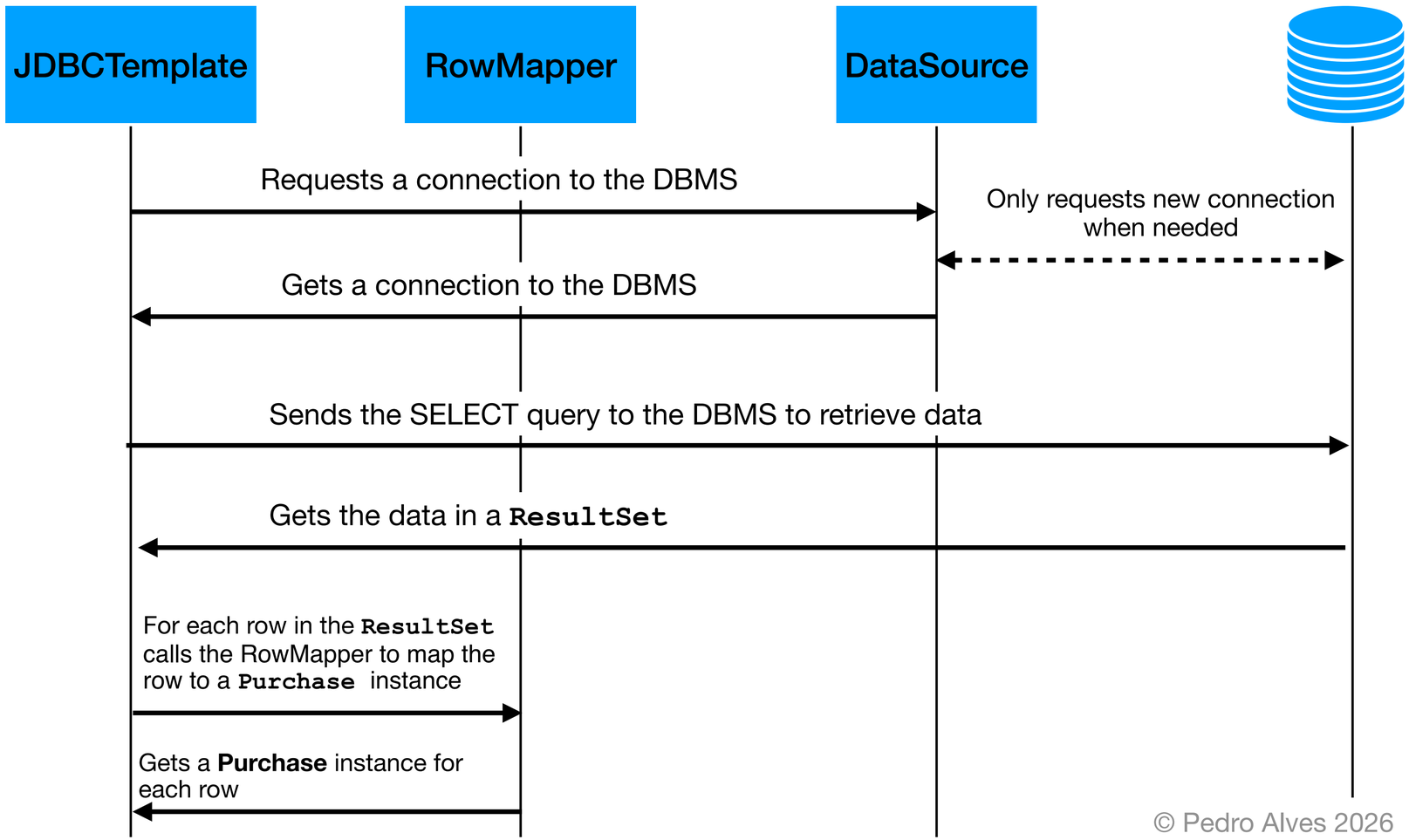
© Pedro Alves 2026

Spring

JDBCTemplate

JDBCTemplate

- Abstraction over JDBC
- Reduces boilerplate code
- Manages connections
- Uses the RowMapper pattern to translate results into objects
- DataSource abstracts the actual DBMS



© Pedro Alves 2026

Spring

JDBCTemplate

JDBCTemplate

- Abstraction over JDBC
- Reduces boilerplate code
- Manages connections
- **Uses the RowMapper pattern to translate results into objects**
- DataSource abstracts the actual DBMS

```
@Repository
class PurchaseRepository(private val jdbcTemplate: JdbcTemplate) {

    fun findAllPurchases(): List<Purchase> {
        val sql = "SELECT * FROM purchase"

        val purchaseRowMapper: RowMapper<Purchase> =
               RowMapper { rs: ResultSet, rowNum: Int ->
                     val purchase = Purchase()
                     purchase.id = rs.getInt("id")
                     purchase.product = rs.getString("product")
                     purchase.price = rs.getBigDecimal("price")
                     purchase
               }

        return jdbcTemplate.query(sql, purchaseRowMapper)
    }
}
```

© Pedro Alves 2026

Spring
JDBCTemplate

Needs this dependency:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

JDBCTemplate can also do updates, deletes, etc…

```
fun updatePurchasePrice(id: String, price: String) {
    jdbcTemplate.update("UPDATE purchase SET price = $price WHERE id = $id")
}

fun deletePurchase(id: String) {
    jdbcTemplate.update("DELETE FROM purchase WHERE id = $id")
}
```

© Pedro Alves 2026

Spring

DataSource

DataSource abstracts the DBMS, just include the dependency of the driver

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

H2 embedded “in-memory” database

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

MySQL database

© Pedro Alves 2026

Spring

In-Memory vs Real Database

Exercise

	In-Memory	Real
Where data lives		
Persistence (yes/no)		
Setup in Spring (automatic/needs configuration)		
Startup speed (slow/medium/fast)		
Good for tests? (yes/no)		
Typical use case		
Example DBMS		

© Pedro Alves 2026

Spring

In-Memory vs Real Database

	In-Memory	Real
Where data lives	In app memory or temp local file	Persistent external db server
Persistence (yes/no)	Usually lost on app stop	Survives restarts
Setup in Spring (automatic/needs configuration)	Auto-configured	Requires connection url, credentials and driver in application.properties
Startup speed (slow/medium/fast)	Very fast	Slower (but still fast)
Good for tests? (yes/no)	Excellent for unit/integration tests	Not convenient (except load tests or production-specific tests)
Typical use case	local experiments; tests	real dev work; production apps
Example DBMS	H2, HSQL, Derby	Oracle, MySQL, Postgres

© Pedro Alves 2026

MySQL

Option 1

Install MySQL

Create a database, application user (*), and password.

```
create database <dbname>;

create user '<user>'@'localhost' identified by '<password>';

grant all privileges on <dbname>.* to ‘<user>'@'localhost';
```

(*) Never access the database using root, even in the dev machine

© Pedro Alves 2026

MySQL

Option 2

Install docker

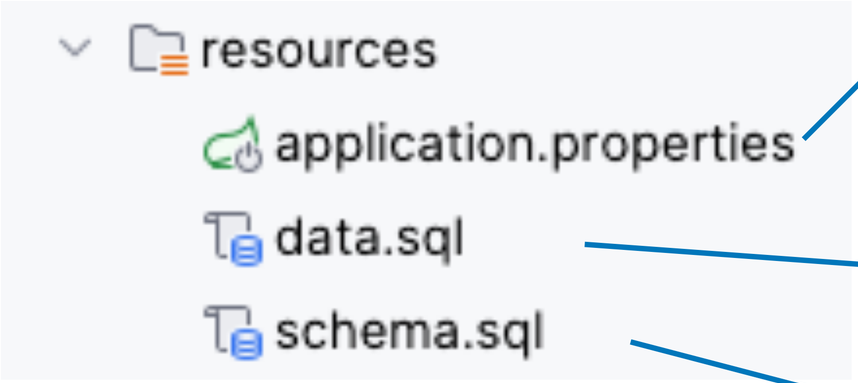
Run docker compose

```yaml
services:
  mysql:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: <rootpass>
      MYSQL_DATABASE: <dbname>
      MYSQL_USER: <user>
      MYSQL_PASSWORD: <password>
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

© Pedro Alves 2026

Spring
JDBCTemplate

Initial database setup




```
spring.datasource.url=jdbc:mysql://localhost:3306/your_db
spring.datasource.username=your_user
spring.datasource.password=your_password
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.sql.init.mode=always
```

Note: For embedded databases this is not needed

```
INSERT INTO purchase (product, price) VALUES (…);
INSERT INTO purchase (product, price) VALUES (…);
```

```
CREATE TABLE IF NOT EXISTS purchase (
    id      INT PRIMARY KEY AUTO_INCREMENT,
    product VARCHAR(255) NOT NULL,
    price   DECIMAL(10, 2) NOT NULL
);
```

© Pedro Alves 2026

Spring

JDBCTemplate

```
fun updatePurchasePrice(id: String, price: String) {
    jdbcTemplate.update("UPDATE purchase SET price = $price WHERE id = $id")
}
```

Is this OK?

© Pedro Alves 2026

Spring

JDBCTemplate

```
fun updatePurchasePrice(id: String, price: String) {
    jdbcTemplate.update("UPDATE purchase SET price = $price WHERE id = $id")
}
```

❌

price: “`0 WHERE 1=1; DROP TABLE purchase; --`”

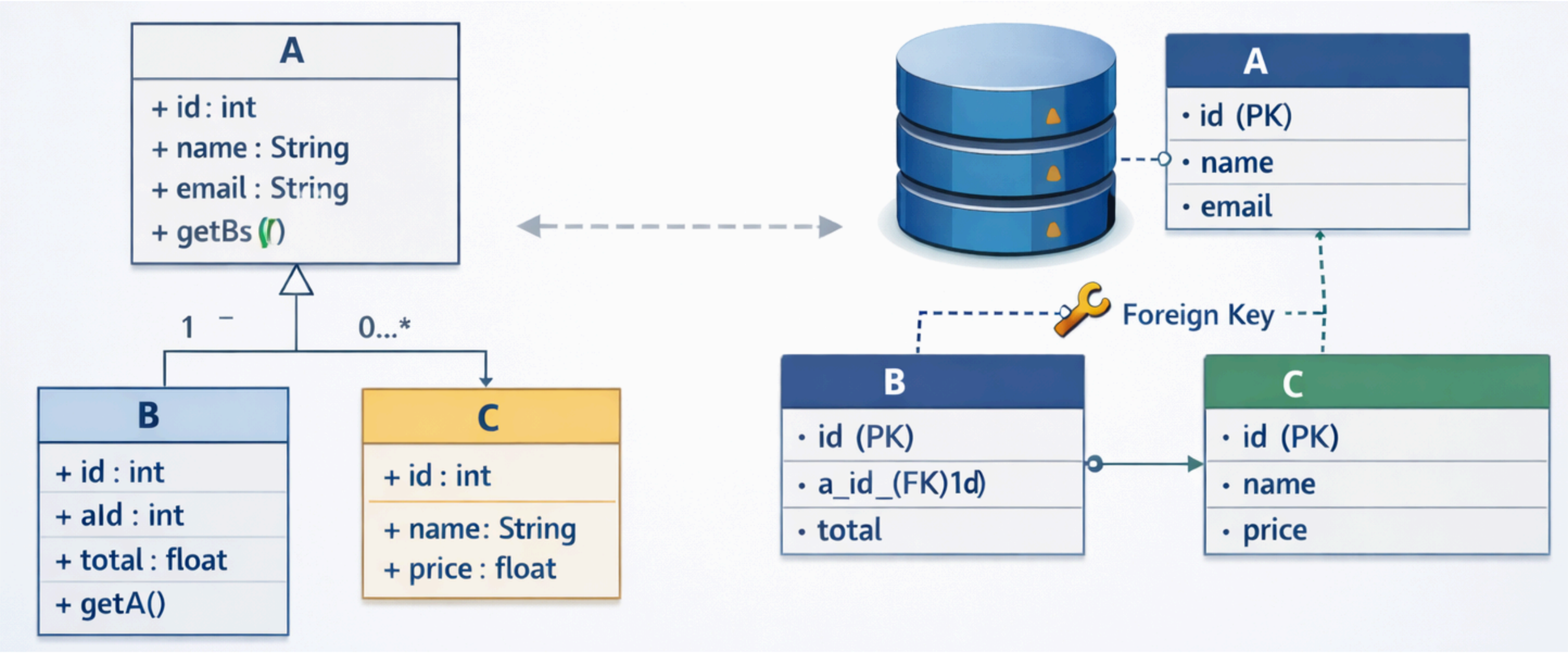
SQL built using String interpolation (or concatenation) is subject to SQL Injection attacks

```
fun updatePurchasePrice(id: String, price: String) {
    jdbcTemplate.update("UPDATE purchase SET price = ? WHERE id = ?", price, id)
}
```

✅

© Pedro Alves 2026

ORM (Object-Relational Mapping)



© Pedro Alves 2026

ORM (Object-Relational Mapping)

- Manages persistence of data in the OO realm
- Abstracts the use of SQL in connection with DMBSs
- Handles the database as a collection of (interconnected) objects
- Protects against most injection attacks
- In Java:
  - Specified by a common and standard API (JPA)
  - Different JPA providers (Hibernate, Eclipselink, …)

© Pedro Alves 2026

ORM Impedance Mismatch

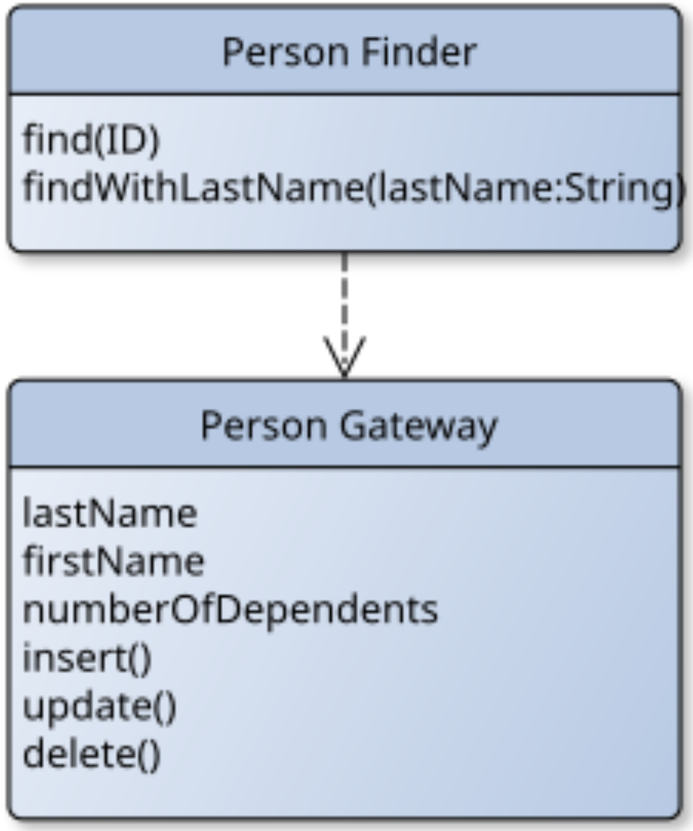
	Objects	Relational
Base entity	class	table
Identity	a == b; a.equals(b)	primary key
Inheritance	Natural relation	Does not exist, idioms are necessary
Accessing data	Through the object interface	SELECT queries and joins
Associations	Unidirectional references through objects' interface	Foreign keys

© Pedro Alves 2026

Row Data Gateway

Row Data Gateway

An object that acts as a gateway to a single record in a data source. There is one instance per row.



https://martinfowler.com/eaaCatalog/rowDataGateway.html

© Pedro Alves 2026

ORM Patterns

Row Data Gateway

```
class UserRowGateway(
    var id: Long,
    var name: String?,
    var email: String?,
    private val jdbcTemplate: JdbcTemplate
) {

    fun insert() {
        jdbcTemplate.update("insert into users(name, email) values (?, ?)", name, email)
    }

    fun update() {
        jdbcTemplate.update("update users set name = ?, email = ? where id = ?", name, email, id)
    }

    fun delete() {
        jdbcTemplate.update("delete from users where id = ?", id)
    }

    companion object {
        fun findById(id: Long, jdbcTemplate: JdbcTemplate): UserRowGateway? {
            return jdbcTemplate.queryForObject(
                "select id, name, email from users where id = ?",
                { rs, _ -> UserRowGateway(rs.getLong("id"), rs.getString("name"), rs.getString("email"), jdbcTemplate) },
                id
            )
        }
    }
}
```

© Pedro Alves 2026

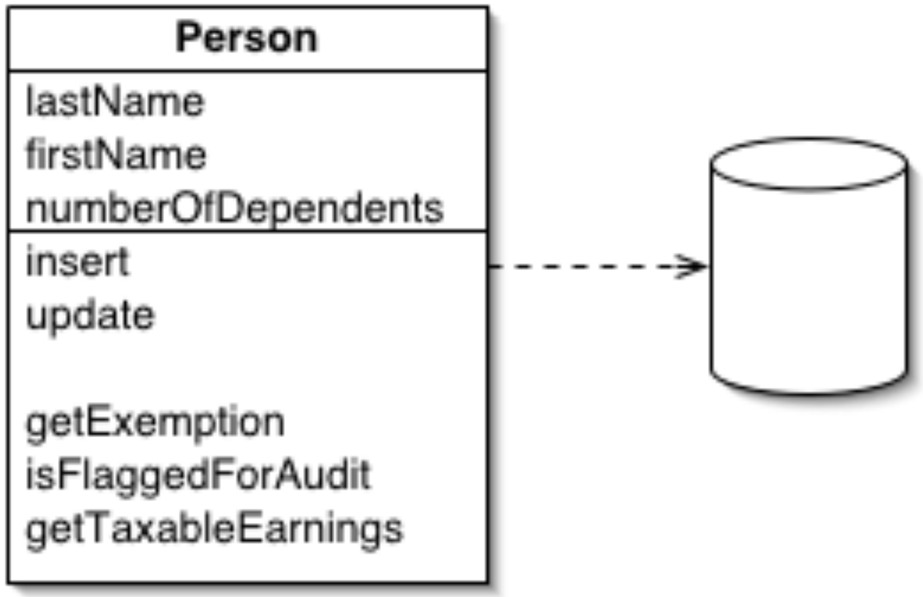
ORM Patterns

Active Record

Active Record

An object that wraps a row in a database table or view, encapsulates the database access, and adds domain logic on that data.

Similar to Row Data Gateway but includes domain logic



https://martinfowler.com/eaaCatalog/activeRecord.html

© Pedro Alves 2026

ORM Patterns

Active Record

Example for Ruby on Rails

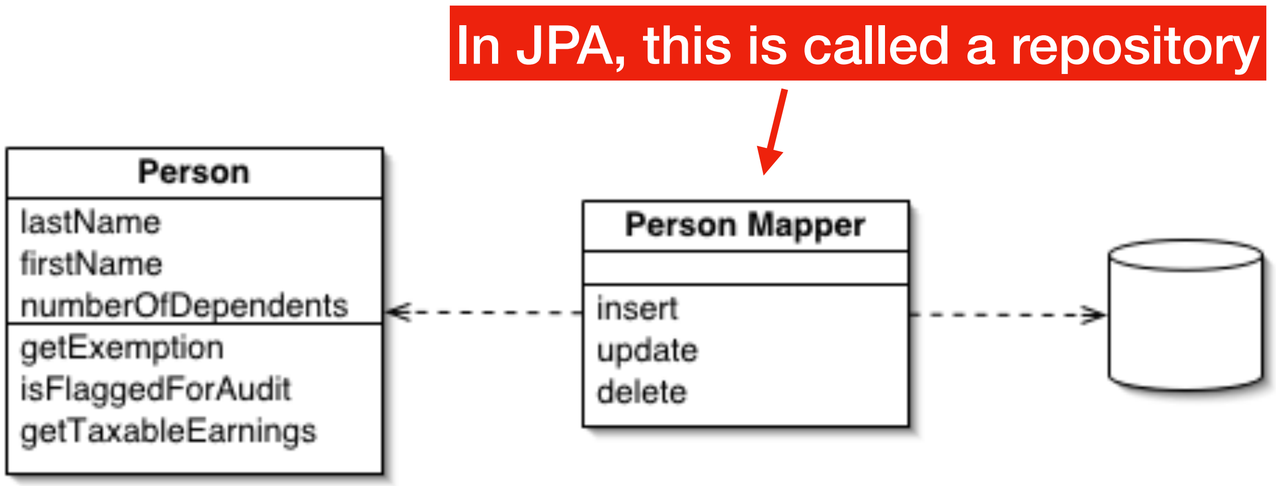
```
class Account < ApplicationRecord
    def withdraw(amount)
        raise “Insufficient funds” if balance < amount
        self.balance -= amount
    end

    def deposit(amount)
        self.balance += amount
    end

    # save(), find(), update(), delete(), etc... inherited from ApplicationRecord
end
```

© Pedro Alves 2026

Data Mapper

Data Mapper

A layer of mappers that moves data between objects and a database while keeping them independent of each other and the mapper itself



https://martinfowler.com/eaaCatalog/dataMapper.html

© Pedro Alves 2026

ORM Patterns

Data Mapper

```
@Entity
class User(
    @Id @GeneratedValue
    var id: Long = 0,

    var name: String,
    var email: String
)
```

```
@Repository
interface UserRepository : JpaRepository<User, Long> {

    // Explicitly redeclared (already inherited)
    override fun save(user: User): User

    fun findByEmail(email: String): User?

    fun findByName(name: String): List<User>
}
```

© Pedro Alves 2026

ORM Patterns

Exercise

	Row Data Gateway	Active Record	Data Mapper
Object represents row (Yes/no)			
Object has save(), delete(), etc.. methods (Yes/no)			
Domain layer separate from persistence layer (Yes/no)			
Domain logic in the object (Yes/no)			
Benefits			

© Pedro Alves 2026

ORM Patterns

	Row Data Gateway	Active Record	Data Mapper
Object represents row (Yes/no)	Yes	Yes	Yes
Object has save(), delete(), etc.. methods (Yes/no)	Yes	Yes	No
Domain layer separate from persistence layer (Yes/no)	No	No	Yes
Domain logic in the object (Yes/no)	No	Yes	Not common, but allowed
Benefits	Simple, direct SQL control	More OO (domain logic encapsulated in the object)	Clean separation of concerns

© Pedro Alves 2026

JPA

- Java Persistence API (Jakarta Persistence since 2019)
- Implements the Data Mapper pattern
- Defines JPQL (Java Persistence Query Language)
- Most popular JPA provider: Hibernate (included in Spring Data JPA)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

© Pedro Alves 2026

JPA

First thing you should do when setting up a JPA application:

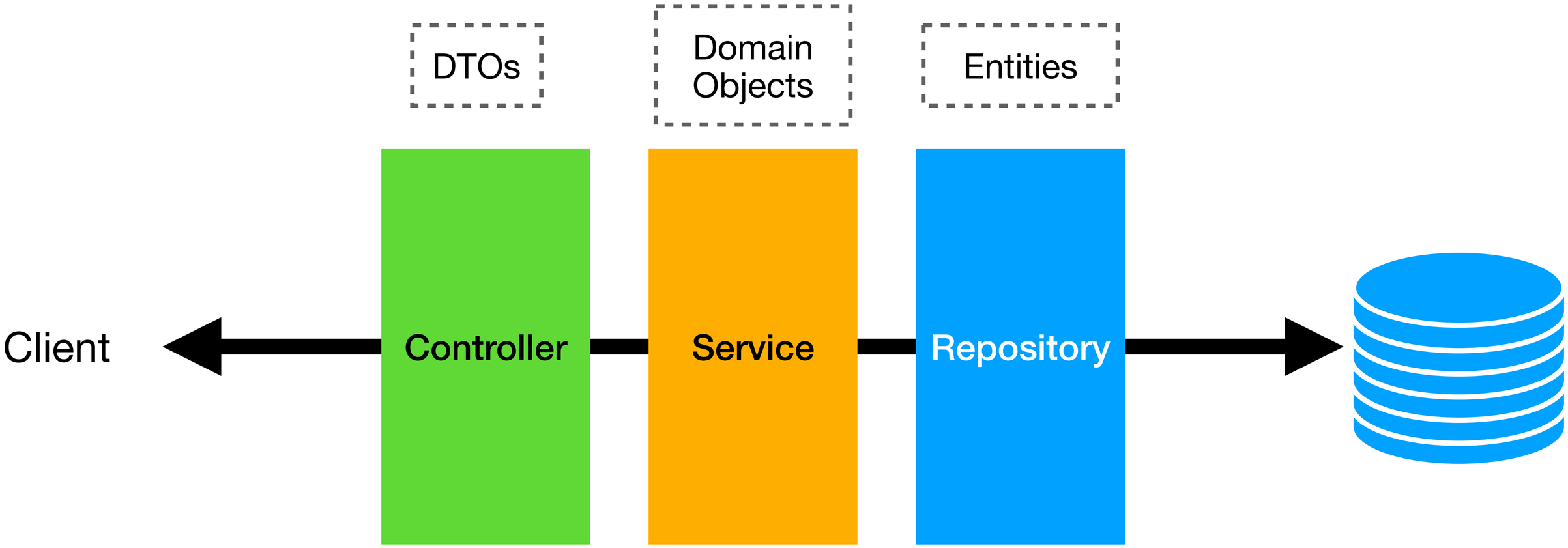
Turn on SQL logging

application.properties

```
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.orm.jdbc.bind=TRACE
```

© Pedro Alves 2026

JPA



© Pedro Alves 2026

JPA

Entity

- Entity is a kind of DTO for databases
- One entity per table (except associative tables for N-N relations) By default, it maps into a table with the same name (snake_case)
- Each attribute is a column in the database (@Column not mandatory)
- Must have an @Id (maps to the primary key)
- Usually don't have behaviour

```
@Entity
class Student {

    @Id
    @GeneratedValue
    (strategy = GenerationType.IDENTITY)
    var id: Long? = null

    @Column(unique = true)
    var studentId: Int? = null

    @Column(length = 100)
    var name: String? = null

    var age: Int? = null
}
```

© Pedro Alves 2026

JPA

Entity

```
@Entity
class Student {

    @Id
    @GeneratedValue
    (strategy = GenerationType.IDENTITY)
    var id: Long? = null

    @Column(unique = true)
    var studentId: Int? = null

    @Column(length = 100)
    var name: String? = null

    var age: Int? = null
}
```

H2

```
create table student (
   id bigint generated by default as identity,
   student_id integer unique,
   name varchar(100),
   age integer,
   primary key (id)
)
```

MySQL

```
create table student (
    age integer,
    student_id integer,
    id bigint not null auto_increment,
    name varchar(100),
    primary key (id)
) engine=InnoDB

alter table student
    add constraint UKlh7d3 unique (student_id)
```

© Pedro Alves 2026

JPA

Entity

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
var id: Long? = null
```

IDs can be managed in three ways:

- **Identity** - The database auto-generates the ID (ex: MySQL `AUTO_INCREMENT`)
- **Sequence** - Uses a database sequence object
- **Table** - Uses a special table to store the next ID (managed by the JPA provider)

© Pedro Alves 2026

JPA

Entity

Entities in Kotlin

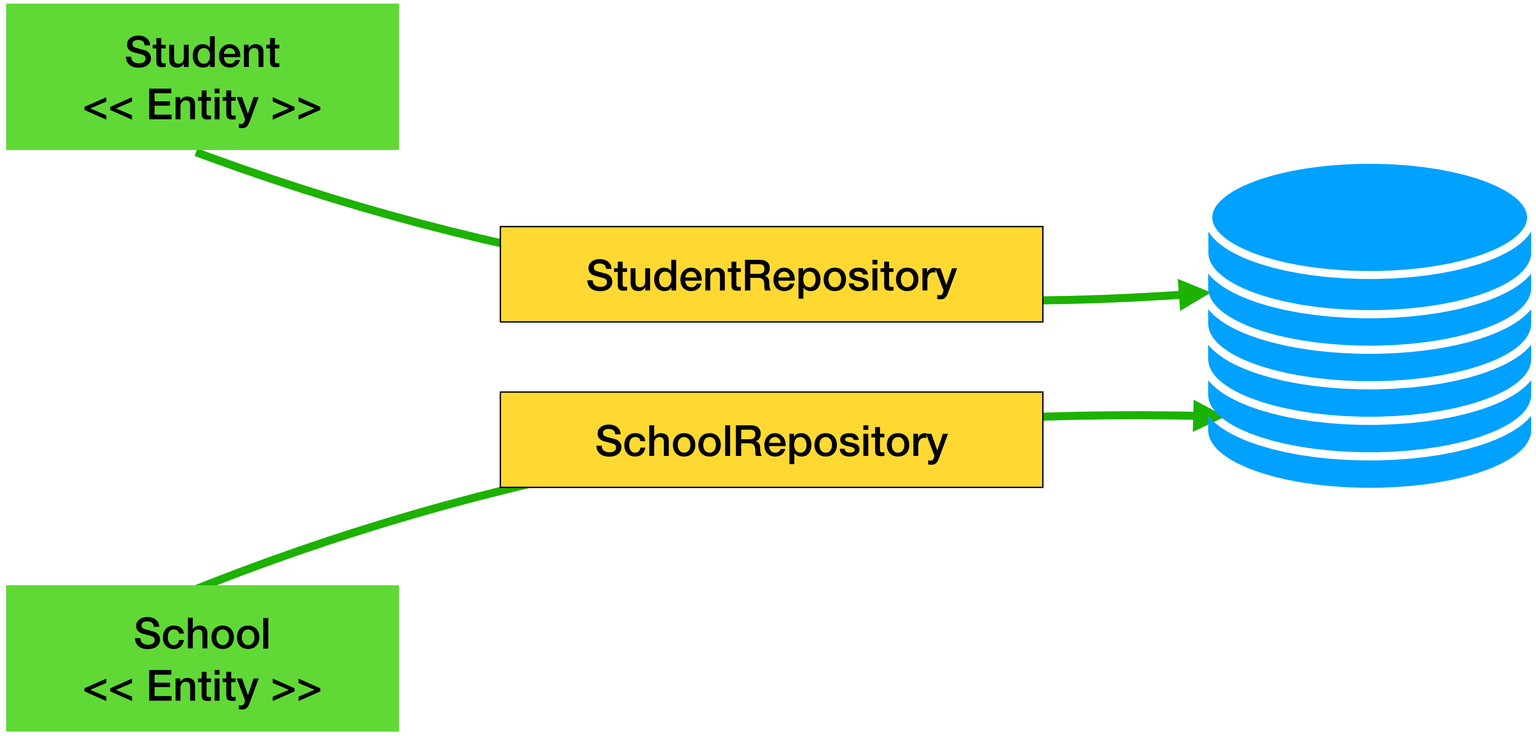
- Must be regular open (non-final) classes (not data classes)
- Must provide a no-arg constructor
- Must have non-final attributes (var and not val)
- Should have nullable attributes (even if they are mandatory)

https://blog.jetbrains.com/idea/2026/01/how-to-avoid-common-pitfalls-with-jpa-and-kotlin/

© Pedro Alves 2026

JPA

Repository

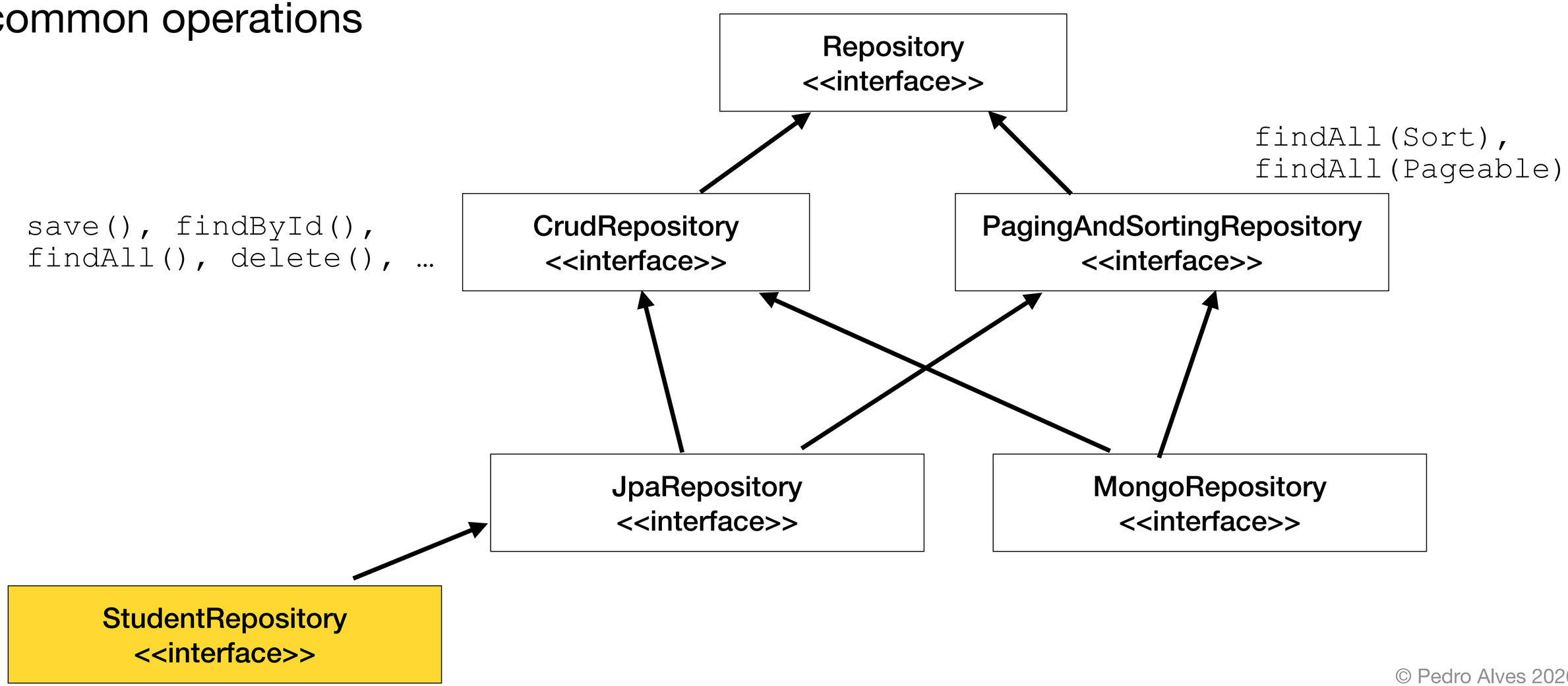


© Pedro Alves 2026

JPA

Repository

Repository is an abstraction over the actual Data Source with common operations



© Pedro Alves 2026

JPA

JpaRepository

```
interface StudentRepository: JpaRepository<Student,Long>
```

```
val student: Student? = studentRepository.findByIdOrNull(1L)
student?.let {
    it.age = 20
    studentRepository.save(it)
}
```

Note: Actually the save() is not necessary, we'll see why later on…

Even an empty interface already gets you the common operations (by inheritance)

© Pedro Alves 2026

JPA

JpaRepository

Add specific operations following a method name convention

findBy<Attribute>

findAllBy<Attribute>

deleteBy<Attribute>

deleteAllBy<Attribute>

existsBy<Attribute>

...

```
interface StudentRepository:
            JpaRepository<Student,Long> {
    fun findByStudentId(studentId: Int): Student?
    fun findAllByAge(age: Int): List<Student>
}
```

© Pedro Alves 2026

JPA

JpaRepository

Combine operations

Keyword	Sample
And	findByLastnameAndFirstname
Or	findByLastnameOrFirstname
Is,Equals	findByFirstname,findByFirstnameIs,findByFirstnameEquals
Between	findByStartDateBetween
LessThan	findByAgeLessThan
LessThanEqual	findByAgeLessThanEqual
GreaterThan	findByAgeGreaterThan
GreaterThanEqual	findByAgeGreaterThanEqual
After	findByStartDateAfter
Before	findByStartDateBefore
IsNull	findByAgeIsNull

© Pedro Alves 2026

JPA

JpaRepository

Create your own queries using JPQL

```
interface StudentRepository: JpaRepository<Student,Long> {

    @Query("SELECT s FROM Student s WHERE s.age BETWEEN :minAge AND :maxAge")
    fun findByAgeRange(minAge: Int, maxAge: Int): List<Student>
}
```

© Pedro Alves 2026

JPA

Paging And Sorting

```
interface StudentRepository: JpaRepository<Student,Long>
```

```
// Sorting only
val sortedByName = studentRepository.findAll(Sort.by(Sort.Direction.ASC, "name"))
sortedByName.forEach { println(it) }


// Paging + sorting: page 0, 5 items per page, sorted by age descending
val pageable = PageRequest.of(0, 5, Sort.by(Sort.Direction.DESC, "age"))
val page: Page<Student> = studentRepository.findAll(pageable)
println("Page ${page.number + 1} of ${page.totalPages}, total students: ${page.totalElements}")
page.content.forEach { println(it) }
```

© Pedro Alves 2026

JPA

Transactions

```
@Service
class StudentService(val studentRepository: StudentRepository) {

    fun replaceStudent(oldId: Long, newStudentId: Int, name: String) {
        studentRepository.deleteById(oldId)

        val s = Student().apply {
            studentId = newStudentId
            this.name = name
            age = 18
        }
        studentRepository.save(s)
    }
}
```

What happens if the application crashes here?

© Pedro Alves 2026

JPA

Transactions

```
@Service
class StudentService(val studentRepository: StudentRepository) {

    fun replaceStudent(oldId: Long, newStudentId: Int, name: String) {
        // start transaction/session
        studentRepository.deleteById(oldId)
        // end transaction/session
```

delete is commited here

```
        val s = Student().apply {
            studentId = newStudentId
            this.name = name
            age = 18
        }
        // start transaction/session
        studentRepository.save(s)
        // end transaction/session
```

insert is commited here

```
    }
}
```

© Pedro Alves 2026

JPA

Transactions

```
@Service
class StudentService(val studentRepository: StudentRepository) {

    @Transactional
    fun replaceStudent(oldId: Long, newStudentId: Int, name: String) {
        // start transaction/session
        studentRepository.deleteById(oldId)

        val s = Student().apply {
            studentId = newStudentId
            this.name = name
            age = 18
        }

        studentRepository.save(s)
        // end transaction/session
    }
}
```

delete and insert are commited here

© Pedro Alves 2026

Associations - One to One

In a bidirectional relationship, one side must own the FK

```
+---------+ 1      0..1 +--------------+
|  book   |-------------|  book_detail |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | synopsis     |
| author  |             | pages        |
+---------+             | FK book_id   |
                        +--------------+
```

This is the owner

© Pedro Alves 2026

JPA

Associations - One to One

```
+---------+ 1      0..1 +--------------+
|  book   |-------------|  book_detail |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | synopsis     |
| author  |             | pages        |
+---------+             | FK book_id   |
                        +--------------+
```

```
@Entity
class Book {

    @Id
    @GeneratedValue(...)
    var id: Long? = null

    var title: String? = null

    var author: String? = null


    var detail: BookDetail? = null
}
```

```
@Entity
class BookDetail {

    @Id
    @GeneratedValue(...)
    var id: Long? = null

    var synopsis: String? = null

    var pages: Int? = null



    var book: Book? = null
}
```

© Pedro Alves 2026

JPA

Associations - One to One

```
+---------+ 1      0..1 +--------------+
|  book   |-------------|  book_detail |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | synopsis     |
| author  |             | pages        |
+---------+             | FK book_id   |
                        +--------------+
```

```java
@Entity
class Book {

    @Id
    @GeneratedValue(...)
    var id: Long? = null

    var title: String? = null

    var author: String? = null

    @OneToOne(mappedBy = "book")
    var detail: BookDetail? = null
}
```

```java
@Entity
class BookDetail {

    @Id
    @GeneratedValue(...)
    var id: Long? = null

    var synopsis: String? = null

    var pages: Int? = null

    @OneToOne
    @JoinColumn(name = "book_id")
    var book: Book? = null
}
```

Note: The @JoinColumn is optional, it's just to make more explicit what will be the FK

© Pedro Alves 2026

JPA

Associations - One to One

```
+---------+ 1      0..1 +--------------+
|  book   |-------------|  book_detail |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | synopsis     |
| author  |             | pages        |
+---------+             | FK book_id   |
                        +--------------+
```

```
@Service
class BookService(val bookRepository: BookRepository,
                  val bookDetailRepository: BookDetailRepository) {

    fun createBook(title: String, author: String, synopsis: String, pages: Int) {

        val book = Book().apply {
            this.title = title
            this.author = author
        }
        bookRepository.save(book)

        val detail = BookDetail().apply {
            this.synopsis = synopsis
            this.pages = pages
        }
        bookDetailRepository.save(detail)
    }
}
```

Creates the 2 rows but doesn’t fill in the FK

© Pedro Alves 2026

JPA

Associations - One to One

```
+---------+ 1      0..1 +--------------+
|  book   |-------------|  book_detail |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | synopsis     |
| author  |             | pages        |
+---------+             | FK book_id   |
                        +--------------+
```

```
@Service
class BookService(val bookRepository: BookRepository,
                  val bookDetailRepository: BookDetailRepository) {

    fun createBook(title: String, author: String, synopsis: String, pages: Int) {

        val book = Book().apply {
            this.title = title
            this.author = author
        }
        bookRepository.save(book)

        val detail = BookDetail().apply {
            this.synopsis = synopsis
            this.pages = pages
        }
        bookDetailRepository.save(detail)

        book.detail = detail
        bookRepository.save(book)
    }
}
```

Does nothing in the DB because it's not the owner of the relationship

© Pedro Alves 2026

JPA

Associations - One to One

```
+---------+ 1      0..1 +--------------+
|  book   |-------------|  book_detail |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | synopsis     |
| author  |             | pages        |
+---------+             | FK book_id   |
                        +--------------+
```

```
@Service
class BookService(val bookRepository: BookRepository,
                  val bookDetailRepository: BookDetailRepository) {

    fun createBook(title: String, author: String, synopsis: String, pages: Int) {

        val book = Book().apply {
            this.title = title
            this.author = author
        }
        bookRepository.save(book)

        val detail = BookDetail().apply {
            this.synopsis = synopsis
            this.pages = pages
        }
        bookDetailRepository.save(detail)

        detail.book = book
        bookDetailRepository.save(detail)
    }
}
```

It works! Setting the value in the owner, fills in the FK

© Pedro Alves 2026

JPA

Associations - One to One

```
+---------+ 1      0..1 +--------------+
|  book   |-------------|  book_detail |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | synopsis     |
| author  |             | pages        |
+---------+             | FK book_id   |
                        +--------------+
```

```
@Service
class BookService(val bookRepository: BookRepository,
                  val bookDetailRepository: BookDetailRepository) {

    fun createBook(title: String, author: String, synopsis: String, pages: Int) {

        val book = Book().apply {
            this.title = title
            this.author = author
        }
        bookRepository.save(book)

        val detail = BookDetail().apply {
            this.synopsis = synopsis
            this.pages = pages
            this.book = book
        }
        bookDetailRepository.save(detail)

    }
}
```

Also works!

© Pedro Alves 2026

JPA

Associations - One to One

```
+---------+ 1      0..1 +--------------+
|  book   |-------------|  book_detail |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | synopsis     |
| author  |             | pages        |
+---------+             | FK book_id   |
                        +--------------+
```

```
@Service
class BookService(val bookRepository: BookRepository,
                  val bookDetailRepository: BookDetailRepository) {

    fun createBook(title: String, author: String, synopsis: String, pages: Int) {

        val book = Book().apply {
            this.title = title
            this.author = author
        }

        val detail = BookDetail().apply {
            this.synopsis = synopsis
            this.pages = pages
            this.book = book
        }

        book.detail = detail

        bookDetailRepository.save(detail)
    }
}
```

Also works!

You have to save using the owner repository

© Pedro Alves 2026

```
+---------+ 1      0..1 +--------------+
|  book   |-------------|  book_detail |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | synopsis     |
| author  |             | pages        |
+---------+             | FK book_id   |
                        +--------------+
```

JPA

Eager Loading

```
interface BookRepository: JpaRepository<Book,Long> {
    fun findByTitle(title: String): Book?
}
```

One-To-One relations automatically fetch both rows (Eager loading)

```
...
val book = bookRepository.findByTitle(title)
```

```
select * from book b1_0 where b1_0.title=?

select * from book_detail bd1_0
left join book b1_0 on b1_0.id=bd1_0.book_id where
bd1_0.book_id=?
```

© Pedro Alves 2026

```
+---------+ 1      0..1 +--------------+
|  book   |-------------|  book_detail |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | synopsis     |
| author  |             | pages        |
+---------+             | FK book_id   |
                        +--------------+
```

JPA

Eager Loading

```
interface BookRepository: JpaRepository<Book,Long> {
    fun findByTitle(title: String): Book?
}


...
val book = bookRepository.findByTitle(title)

println(book.detail)
```

One-To-One relations automatically fetch both rows (Eager loading)

No queries executed, it’s already in memory

© Pedro Alves 2026

JPA

Associations - One to Many

```
+----------+ 1      0..N +---------------+
|  book    |--------------|  review       |
+----------+              +---------------+
| PK id    |              | PK id         |
| title    |              | comment       |
| author   |              | FK book_id    |
+----------+              +---------------+
```

This is the owner

© Pedro Alves 2026

JPA

Associations - One to Many

```
+---------+ 1      0..N +--------------+
|  book   |-------------|  review      |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | comment      |
| author  |             | FK book_id   |
+---------+             +--------------+
```

```
@Entity
class Book {

    @Id
    @GeneratedValue(...)
    var id: Long? = null

    ...

    @OneToOne(mappedBy = "book")
    var detail: BookDetail? = null


    val reviews: MutableList<Review> =
                                mutableListOf()

}
```

```
@Entity
class Review {

    @Id
    @GeneratedValue(...)
    val id: Long? = null

    var comment: String? = null



    var book: Book? = null
}
```

© Pedro Alves 2026

JPA

Associations - One to Many

```
+---------+ 1      0..N +--------------+
|  book   |-------------|  review      |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | comment      |
| author  |             | FK book_id   |
+---------+             +--------------+
```

```
@Entity
class Book {

    @Id
    @GeneratedValue(...)
    var id: Long? = null

    ...

    @OneToOne(mappedBy = "book")
    var detail: BookDetail? = null

    @OneToMany(mappedBy = "book")
    val reviews: MutableList<Review> =
                                mutableListOf()

}
```

```
@Entity
class Review {

    @Id
    @GeneratedValue(...)
    val id: Long? = null

    var comment: String? = null

    @ManyToOne
    @JoinColumn(name = "book_id")
    var book: Book? = null
}
```

© Pedro Alves 2026

JPA

Associations - One to Many

```
+---------+ 1      0..N +--------------+
|  book   |-------------|  review      |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | comment      |
| author  |             | FK book_id   |
+---------+             +--------------+
```

```
@Service
class BookService(
    val bookRepository: BookRepository,
    val reviewRepository: ReviewRepository
) {

    fun addReview(bookId: Long, comment: String) {
        val book = bookRepository.findById(bookId).orElseThrow()

        val review = Review().apply {
            this.comment = comment
            this.book = book
        }

        reviewRepository.save(review)
    }
}
```

The relation must be created in the owner (again)

Note: If the book doesn't exist, it throws a NoSuchElementException

© Pedro Alves 2026

Exercise

Spot the errors

```
1.  @Entity
2.  @Table(name = "orders")
3.  data class Orders (
4.
5.      var id: Long?,
6.
7.      @Column(nullable = false)
8.      val customerName: String,
9.
10.     var totalAmount: Double?,
11.
12.     @OneToOne
13.     @JoinColumn(name = "customer_id")
14.     var customer: Customer?,
15.
16.     @OneToMany(mappedBy = "order")
17.     @JoinColumn(name = "order_id")
18.     var items: MutableList<OrderItem> = mutableListOf(),
19. )
```

© Pedro Alves 2026

Resolution

```
1.  @Entity
2.  @Table(name = "orders")
3.  data class Orders (
4.
5.      var id: Long?,
6.
7.      @Column(nullable = false)
8.      val customerName: String,
9.
10.     var totalAmount: Double?,
11.
12.     @OneToOne
13.     @JoinColumn(name = "customer_id")
14.     var customer: Customer?,
15.
16.     @OneToMany(mappedBy = "order")
17.     @JoinColumn(name = "order_id")
18.     var items: MutableList<OrderItem> = mutableListOf(),
19. )
```

entities should be singular; should not be data class

missing @Id and @GeneratedValue

should use var, all entity attributes should be var

should be @ManyToOne

if it was @ManyToOne this would be correct

the @JoinColumn is in the table with FK (OrderItem)

© Pedro Alves 2026

```
+---------+ 1      0..N +--------------+
|  book   |-------------|  review      |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | comment      |
| author  |             | FK book_id   |
+---------+             +--------------+
```

JPA

Lazy Loading

```
interface BookRepository: JpaRepository<Book,Long> {
    fun findByTitle(title: String): Book?
}


...
val book = bookRepository.findByTitle(title)
```

A one-to-many relationship does not fetch the “many” side by default (lazy loading)

```
select * from book b1_0 where b1_0.title=?
```

(doesn’t fetch the reviews of this book)

© Pedro Alves 2026

```
+---------+ 1      0..N +--------------+
|  book   |-------------|  review      |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | comment      |
| author  |             | FK book_id   |
+---------+             +--------------+
```

JPA

Lazy Loading

```
interface BookRepository: JpaRepository<Book,Long> {
    fun findByTitle(title: String): Book?
}


...
val book = bookRepository.findByTitle(title)

val reviews = book?.reviews
```

(still doesn’t fetch the reviews of this book)

© Pedro Alves 2026

JPA

Lazy Loading

```
+---------+ 1      0..N +--------------+
|  book   |-------------|  review      |
+---------+             +--------------+
| PK id   |             | PK id        |
| title   |             | comment      |
| author  |             | FK book_id   |
+---------+             +--------------+
```

```
interface BookRepository: JpaRepository<Book,Long> {
    fun findByTitle(title: String): Book?
}
```

```
...
val book = bookRepository.findByTitle(title)

val reviews = book?.reviews

println("reviews = ${reviews}")
```

fetches when it's accessed for the first time

```
select * from review r1_0 where r1_0.book_id=?
```

© Pedro Alves 2026

JPA

Lazy Loading

```
@Entity
class Book {

    ...

    @OneToMany(mappedBy = "book")
    val reviews: ArrayList<Review> = ArrayList()
}
```

```
...
val book = bookRepository.findByTitle(title)

val reviews = book?.reviews

println(reviews!!::class.java)
```

```
class org.hibernate.collection.spi.PersistentBag
```

© Pedro Alves 2026

JPA

Lazy Loading

In runtime, Spring replaces (through reflection) the value of reviews

```
@Entity
class Book {

    ...

    @OneToMany(mappedBy = "book")
    val reviews: ArrayList<Review> = ArrayList()
}
```

PersistentBag(session(*))

- Extends List
- The first operation triggers a SELECT using the session

(*) session is a database session/transaction

© Pedro Alves 2026

JPA

Lazy Loading

```
fun printReviews(bookTitle: String) {
    val book = bookRepository.findByTitle(bookTitle)
    val reviews = book?.reviews
    println("reviews = ${reviews}")
}
```

```
failed to lazily initialize a collection of role:
pt.unl.fct.iadi.jpaexperiments.entity.Book.reviews: could not
initialize proxy - no Session
```

© Pedro Alves 2026

JPA

Lazy Loading

```
fun printReviews(bookTitle: String) {
    // Spring Data's findByTitle has @Transactional(readOnly=true) internally
    // ┌─ session opens
    // ├─ SELECT * FROM book WHERE title = ?
    // ├─ Book is instantiated, reviews field replaced with PersistentBag(session=X)
    // └─ session closes, Book becomes DETACHED, PersistentBag.session → closed
    val book = bookRepository.findByTitle(bookTitle)

    // just copies a reference to the PersistentBag — no session needed, safe
    val reviews = book?.reviews

    // calls reviews.toString() → PersistentBag tries to use its session reference
    // LazyInitializationException: session is already closed
    println("reviews = ${reviews}")

}
```

© Pedro Alves 2026

JPA

Lazy Loading

```
@Transactional
fun printReviews(bookTitle: String) {
    // ┌─ session opens (Spring AOP intercepts before method body runs)
    // ├─ SELECT * FROM book WHERE title = ?
    // ├─ Book is instantiated, reviews field replaced with PersistentBag(session=X)
    // └─ session remains open (outer @Transactional is still active)
    val book = bookRepository.findByTitle(bookTitle)

    // just copies a reference to the PersistentBag — no session needed, safe
    val reviews = book?.reviews

    // calls reviews.toString() → PersistentBag uses the live session
    // ├─ SELECT * FROM review WHERE book_id = ?
    // └─ collection initialized successfully
    println("reviews = ${reviews}")

} // session closes here, when the method returns
```

© Pedro Alves 2026

JPA

@Transactional

```
@Service
class BookService(...) {

    fun printReviews(bookTitle: String) {
        // ...
    }
}
```

```
@Service
class BookService(...) {

    @Transactional
    fun printReviews(bookTitle: String) {
        // ...
    }
}
```

```
println(bookService::class.java)
```

BookService

BookService$$SpringCGLIB$$0

© Pedro Alves 2026

JPA

@Transactional

`BookService` → `BookService$$SpringCGLIB$$0`

CGLIB (Code Generation Library) is a bytecode generation library that Spring uses to create subclass-based proxies at runtime.

```
// What CGLIB generates at runtime (simplified)
class BookService$$SpringCGLIB$$0 : BookService() {
    override fun printReviews(...) {
        transactionInterceptor.before()   // open session, begin tx
        super.printReviews(...)           // your actual code
        transactionInterceptor.after()    // commit, close session
    }
}
```

© Pedro Alves 2026

JPA

Lazy vs Eager Loading

```
@Entity
class Book {

    // ...

    @OneToMany(mappedBy = "book",
               fetch = FetchType.LAZY)
    val reviews: MutableList<Review> = ArrayList()
}
```

BookService

```
@Transactional
fun printReviews(bookTitle: String) {
    val book = bookRepository.findByTitle(bookTitle)
    val reviews = book?.reviews
    println("reviews = ${reviews}")
}
```

```
@Entity
class Book {

    // ...

    @OneToMany(mappedBy = "book",
               fetch = FetchType.EAGER)
    val reviews: MutableList<Review> = ArrayList()
}
```

BookService

```
fun printReviews(bookTitle: String) {
    val book = bookRepository.findByTitle(bookTitle)
    val reviews = book?.reviews
    println("reviews = ${reviews}")
}
```

© Pedro Alves 2026

JPA

Eager vs Lazy Loading

Eager Loading

- Loads the related entities immediately with the parent entity
- Default for @ManyToOne and @OneToOne
- Can cause unnecessary high memory consumption

Lazy Loading

- Related entities are loaded only when accessed
- Default in @OneToMany and @ManyToMany relationships
- Needs @Transactional
- Faster initial query
- More memory efficient

© Pedro Alves 2026

Associations - Many to Many

```
+----------+ 1      N +----------------+ N      1 +---------------+
|  book    |-----------|  book_author   |-----------|    author     |
+----------+           +----------------+           +---------------+
| PK id    |           | FK book_id     |           | PK id         |
| title    |           | FK author_id   |           | name          |
+----------+           +----------------+           +---------------+
```

This is the owner

In Many-to-Many, the owner is the table that is usually first inserted. In this case, books are inserted first, and then their authors

© Pedro Alves 2026

JPA

Associations - Many to Many

```
+---------+ 1      N +---------------+ N      1 +--------------+
|  book   |----------|  book_author  |----------|    author    |
+---------+          +---------------+          +--------------+
| PK id   |          | FK book_id    |          | PK id        |
| title   |          | FK author_id  |          | name         |
+---------+          +---------------+          +--------------+
```

```
@Entity
class Book {

    @Id
    @GeneratedValue(...)
    var id: Long? = null

    var title: String? = null





    val authors: MutableList<Author> = ArrayList()
}
```

```
@Entity
class Author {

    @Id
    @GeneratedValue(...)
    var id: Long? = null

    var name: String? = null


   val books: MutableList<Book> = ArrayList()
}
```

© Pedro Alves 2026

JPA

Associations - Many to Many

```
+---------+ 1      N +---------------+ N      1 +--------------+
|  book   |----------|  book_author  |----------|    author    |
+---------+          +---------------+          +--------------+
| PK id   |          | FK book_id    |          | PK id        |
| title   |          | FK author_id  |          | name         |
+---------+          +---------------+          +--------------+
```

```
@Entity
class Book {

    @Id
    @GeneratedValue(...)
    var id: Long? = null

    var title: String? = null

    @ManyToMany
    @JoinTable(
        name = "book_author",
        joinColumns = [JoinColumn(name = "book_id")],
        inverseJoinColumns = [JoinColumn(name = "author_id")]
    )
    val authors: MutableList<Author> = ArrayList()
}
```

```
@Entity
class Author {

    @Id
    @GeneratedValue(...)
    var id: Long? = null

    var name: String? = null

    @ManyToMany(mappedBy = "authors")
    val books: MutableList<Book> = ArrayList()
}
```

© Pedro Alves 2026

JPA

Associations - Many to Many

```
+---------+ 1      N +---------------+ N      1 +--------------+
|  book   |----------|  book_author  |----------|    author    |
+---------+          +---------------+          +--------------+
| PK id   |          | FK book_id    |          | PK id        |
| title   |          | FK author_id  |          | name         |
+---------+          +---------------+          +--------------+
```

```
@Transactional
fun addAuthor(bookTitle: String, authorName: String) {
    val book = bookRepository.findByTitle(bookTitle) ?: throw NoSuchElementException()
    val author = authorRepository.findByName(authorName) ?: Author().apply {
        this.name = authorName
        authorRepository.save(this)
    }
    book.authors.add(author)
    bookRepository.save(book)
}
```

© Pedro Alves 2026

JPA

Associations - Many to Many

```
+---------+ 1      N +---------------+ N      1 +--------------+
|  book   |----------|  book_author  |----------|    author    |
+---------+          +---------------+          +--------------+
| PK id   |          | FK book_id    |          | PK id        |
| title   |          | FK author_id  |          | name         |
+---------+          +---------------+          +--------------+
```

you can fetch from both sides

```
@Transactional
fun printBookAndAuthors(bookTitle: String) {
    val book = bookRepository.findByTitle(bookTitle)
    println("book = ${book}")
    println("book.authors = ${book.authors}")
}

@Transactional
fun printAuthorAndBooks(authorName: String) {
    val author = authorRepository.findByName(authorName)
    println("author = ${author}")
    println("author.books = ${author.books}")
}
```

© Pedro Alves 2026

```
+---------+ 1      N +---------------+ N      1 +--------------+
|  book   |----------|  book_author  |----------|    author    |
+---------+          +---------------+          +--------------+
| PK id   |          | FK book_id    |          | PK id        |
| title   |          | FK author_id  |          | name         |
+---------+          +---------------+          +--------------+
```

JPA

Associations - Many to Many

@ManyToMany relations are lazy fetch by default

```
@Transactional
fun printBookAndAuthors(bookTitle: String) {
    val book = bookRepository.findByTitle(bookTitle)
    println("book = ${book}")
    println("book.authors = ${book.authors}")
}
```

fetches only the book

fetches the authors

```
@Transactional
fun printAuthorAndBooks(authorName: String) {
    val author = authorRepository.findByName(authorName)
    println("author = ${author}")
    println("author.books = ${author.books}")
}
```

fetches only the author

fetches the books

© Pedro Alves 2026

JPA

N+1 query problem

Get all books and their reviews

```
val books = bookRepository.findAll()
for (book in books) {
    for (review in book.reviews) {
        println("review = ${review}")
    }
}
```

```
SELECT * FROM book
SELECT * FROM review WHERE book_id=1
SELECT * FROM review WHERE book_id=2
SELECT * FROM review WHERE book_id=3
...
```

It should be

```
SELECT b.*, r.* FROM book b JOIN review r ON r.book_id = b.id
```

© Pedro Alves 2026

JPA

Prefetching

Instead of using global fetching strategies, one can define how objects/collections related to one particular entity are loaded to memory

```
interface BookRepository: JpaRepository<Book,Long> {
    @Query("SELECT b FROM Book b JOIN FETCH b.reviews")
    fun findAllWithReviews(): List<Book>
}
```

```
SELECT b.*, r.* FROM book b JOIN review r ON r.book_id = b.id
```

© Pedro Alves 2026

JPA

Custom queries

ORM relations can produce a large number of queries… which can be optimized by means of a single query being dispatched to the DB

```
@Query(
"""
select distinct s from Student s
join s.courses en
join en.course c
where c.name = :name
"""
)
fun findByCourseName(name: String): List<Student>
```

© Pedro Alves 2026

JPA

Cascading

If I do something to Parent, should it also happen to its children?

```
@Entity
class Book {
   ...

   @OneToMany(mappedBy = "book", cascade = [CascadeType.ALL])
   val reviews: MutableList<Review> = ArrayList()
}
```

```
fun deleteBook(bookTitle: String) {
    val book = bookRepository.findByTitle(bookTitle) ?: return
    bookRepository.delete(book)
}
```

This will delete the book and all its reviews

© Pedro Alves 2026

JPA

Enumeration vs Separate Table

```
+-----------+
|  book     |
+-----------+
| PK id     |
| title     |
| category  |
+-----------+
```

```
enum class BookCategory {
    FICTION, NON_FICTION, SCIENCE
}

@Entity
class Book {

    ...

    @Enumerated(EnumType.STRING)
    var category: BookCategory? = null
}
```

```
+----------------+ N         1 +--------------+
|  book          |-------------|  category    |
+----------------+             +--------------+
| PK id          |             | PK id        |
| title          |             | name         |
| FK category_id |             +--------------+
+----------------+
```

```
@Entity
class Book {
    ...

    @ManyToOne
    @JoinColumn(name = "category_id")
    var category: BookCategory? = null
}

@Entity
class BookCategory {

    @Id
    @GeneratedValue(...)
    var id: Long? = null

    var name: String? = null
}
```

Note: ManyToOne relations are eagerly fetched

© Pedro Alves 2026

JPA

Enumeration vs Separate Table

	Enumeration	Separate table
Where defined	Kotlin enum	Database table
Add new entry	requires code change + deploy	just insert row
Database normalization	Not normalized	Fully normalized
Validation	Compile-time safety	Runtime
Performance	Fast	Requires join
Best for	Fixed small sets	Evolving domain data

© Pedro Alves 2026

JPA

Database creation

1. Connect to a DBMS as root
2. Create the database schema
3. Create the user/pass to access that schema
4. Create the tables and constraints
5. Populate with initial data

© Pedro Alves 2026

JPA

Database creation

1. **Connect to a DBMS as root**
2. **Create the database schema**
3. **Create the user/pass to access that schema**
4. Create the tables and constraints
5. Populate with initial data

We already talked about this in the beginning of the lesson:

Option 1 - Install MySQL, run script

Option 2 - Install docker, run docker compose

© Pedro Alves 2026

JPA

Database creation

1. Connect to a DBMS as root
2. Create the database schema
3. Create the user/pass to access that schema
4. **Create the tables and constraints**
5. Populate with initial data

application.properties

```
spring.jpa.hibernate.ddl-auto=create
```

On startup:

- all entities are translated to create table xxx
- all foreign keys and constraint are created
- note: this is a destructive operation, the database is recreated, all data is lost

© Pedro Alves 2026

JPA

Database creation

1. Connect to a DBMS as root
2. Create the database schema
3. Create the user/pass to access that schema
4. Create the tables and constraints
5. **Populate with initial data**

On startup:

after creating tables, execute the sql instructions in data.sql

application.properties

```
spring.jpa.defer-datasource-initialization=true
spring.sql.init.mode=always
```

+

src/main/resources/data.sql

```
INSERT INTO student (id, name) VALUES (1, 'Alic
INSERT INTO student (id, name) VALUES (2, 'Bob');
```

Problems:

- SQL specific to a certain DBMS
- Auto-assigned IDs?
- Doesn't survive refactorings
- What if data is already there?

© Pedro Alves 2026

JPA

Database creation

1. Connect to a DBMS as root
2. Create the database schema
3. Create the user/pass to access that schema
4. Create the tables and constraints
5. **Populate with initial data**

Spring components that extend ApplicationRunner run automatically on startup

```
@Component
class DataInitializer(
    private val studentRepository: StudentRepository,
) : ApplicationRunner {

    override fun run(args: ApplicationArguments) {
        if (studentRepository.count() > 0) return

        studentRepository.save(Student(name = "Alice"))
        studentRepository.save(Student(name = “Bob”))
    }
}
```

Benefits:

- Works with any DBMS
- Auto-assigned IDs work fine
- Affected by refactorings
- Checks if data is already there

© Pedro Alves 2026

JPA

Database evolution

Unlike application code, databases cannot be easily replaced when deploying a new software version to production

v1

```
@Entity
class Author {
    @Id
    var id: Long? = null

    var name: String? = null
}
```

v2

```
@Entity
class Author {
    @Id
    var id: Long? = null

    var firstName: String? = null
    var surName: String? = null
}
```

id	name
1	Pedro Alves
2	Cristina Neves
3	João Silva

????

© Pedro Alves 2026

JPA

Database evolution

If the changes are non-destructive (example, adding a new nullable column), you can use this flag:

application.properties

```
spring.jpa.hibernate.ddl-auto=update
```

This is fine in the developer machine.

If needed, just drop the database and start again

© Pedro Alves 2026

JPA

Database evolution - migrations

1. Every schema change is stored in a versioned sql file:

```
V1__create_users_table.sql
V2__add_email_column.sql
V3__insert_default_data.sql
```

2. There is a special table in the database that controls which versions have already been applied

3. On startup, the application runs pending migrations in order

Spring doesn’t include this out-of-the box. Available options: Flyway, Liquibase

Django and Ruby On Rails include a migration tool

© Pedro Alves 2026

JPA

```
@GetMapping("/{id}")
fun detail(@PathVariable id: Long,
           model: ModelMap): String {
    model["club"] = clubService.findById(id)
    return "clubs/detail"
}
```

```
interface ClubRepository : JpaRepository<Club, Long>
```

```
@Service
class ClubService(val clubRepository: ClubRepository) {

    fun findById(id: Long): Club =
        clubRepository.findById(id).orElseThrow()

}
```

© Pedro Alves 2026

JPA

Exceptions

```
@Service
class ClubService(val clubRepository: ClubRepository) {

    fun findById(id: Long): Club =
        clubRepository.findById(id).orElseThrow()
}
```

Services should throw Exception on errors. If there is no record, throw `NoSuchElementException`

```
@ControllerAdvice
class GlobalControllerAdvice {

    @ExceptionHandler(NoSuchElementException::class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    fun handleNotFound(ex: NoSuchElementException, model: Model): String {
        model.addAttribute("message", ex.message ?: "Not found")
        return "error/404"
    }
}
```

© Pedro Alves 2026

Exercise

What is the output of the is program, including sql?

```
@Entity
class Student {
    ...
    var name: String? = null

    @OneToOne(mappedBy = "student")
    var info: StudentInfo? = null

    @ManyToOne
    var course: Course? = null
}
```

```
@Entity
class StudentInfo {
    ...
    var address: String? = null

    @OneToOne
    var student: Student? = null
}
```

```
@Entity
class Course {
   ...
   var name: String? = null

   @OneToMany(mappedBy = "course")
   val students: MutableList<Student> = ArrayList()
}
```

```
val courses = courseRepository.findAll()
for (course in courses) {
    println("Course: ${course.name}")
    for (student in course.students) {
        println("Student Name: ${student.name}")
        println("Student Address: " +
                "${student.info?.address}")
    }
}
```

```
schoolService.createStudent(1001, "Alice","123 Main St")
schoolService.createStudent(1002, "Bob","456 Oak Ave")
schoolService.createStudent(1003, "Claire","789 Tor Ave")

schoolService.createCourse("Databases")
schoolService.createCourse("Algorithms")

schoolService.enroll(1001, "Databases")
schoolService.enroll(1002, "Databases")
schoolService.enroll(1003, "Algorithms")
```

© Pedro Alves 2026

Resolution

```
@Entity
class Student {
    ...
    var name: String? = null

    @OneToO
    var inf

    @ManyTo
    var cou
}
```

```
@Entity
class StudentInfo {
    ...
    var address: String? = null

    t? = null
```

```
val courses = courseRepository.findAll()
for (course in courses) {
    println("Course: ${course.name}")
    for (student in course.students) {
        println("Student Name: ${student.name}")
        println("Student Address: " +
                "${student.info?.address}")
    }
}
```

```
@Entity
class Cours
   ...
   var name: String? = null

   @OneToMany(mappedBy = "course")
   val students: MutableList<Student> = ArrayList()
}
```

```
schoolService.createStudent(1001, "Alice","123 Main St")
schoolService.createStudent(1002, "Bob","456 Oak Ave")
schoolService.createStudent(1003, "Claire","789 Tor Ave")

schoolService.createCourse("Databases")
schoolService.createCourse("Algorithms")

schoolService.enroll(1001, "Databases")
schoolService.enroll(1002, "Databases")
schoolService.enroll(1003, "Algorithms")
```

```
select * from course
Course: Databases
select * from student s left join
student_info where s.course_id=1
Student Name: Alice
Student Address: 123 Main St
Student Name: Bob
Student Address: 456 Oak Ave
Course: Algorithms
select * from student s left join
student_info where s.course_id=2
Student Name: Claire
Student Address: 789 Tor Ave
```

© Pedro Alves 2026